

Session 17 (AIML) -Assignment Question

- **Name of Student:** Shruti Dnyaneshwar Darwatkar
- **Domain Name:** AIML Development
- **College Name:** Zeal Polytechnic, Narhe
- **Branch Name:** AIML
- **Github url:** https://github.com/shrutidarwatkar-sketch/Task_ITR.git

Q1. Load the HeartDisease dataset using pandas and display the first 10 rows. Also check the shape and basic information (info ()) of the dataset.

Program screenshot:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
from google.colab import files
```

```
uploaded = files.upload()
```

```
... Choose Files heart.csv
heart.csv(text/csv) - 35921 bytes, last modified: 7/9/2026 - 100% done
Saving heart.csv to heart.csv
```

```
#Q1
print("First 10 Rows:")
print(df.head(10))

print("\nShape of Dataset:")
print(df.shape)

print("\nDataset Information:")
df.info()
```

... First 10 Rows:

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	\
0	40	M	ATA	140	289	0	Normal	172	
1	49	F	NAP	160	180	0	Normal	156	
2	37	M	ATA	130	283	0	ST	98	
3	48	F	ASY	138	214	0	Normal	108	
4	54	M	NAP	150	195	0	Normal	122	
5	39	M	NAP	120	339	0	Normal	170	
6	45	F	ATA	130	237	0	Normal	170	
7	54	M	ATA	110	208	0	Normal	142	
8	37	M	ASY	140	207	0	Normal	130	
9	48	F	ATA	120	284	0	Normal	120	

	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	N	0.0	Up	0
1	N	1.0	Flat	1
2	N	0.0	Up	0
3	Y	1.5	Flat	1
4	N	0.0	Up	0
5	N	0.0	Up	0
6	N	0.0	Up	0
7	N	0.0	Up	0
8	Y	1.5	Flat	1
9	N	0.0	Up	0

Shape of Dataset:
(918, 12)

Dataset Information:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 918 entries, 0 to 917

Data columns (total 12 columns):

#	Column	Non-Null Count	Dtype
0	Age	918 non-null	int64
1	Sex	918 non-null	object
2	ChestPainType	918 non-null	object
3	RestingBP	918 non-null	int64
4	Cholesterol	918 non-null	int64
5	FastingBS	918 non-null	int64
6	RestingECG	918 non-null	object
7	MaxHR	918 non-null	int64
8	ExerciseAngina	918 non-null	object
9	Oldpeak	918 non-null	float64
10	ST_Slope	918 non-null	object
11	HeartDisease	918 non-null	int64

dtypes: float64(1), int64(6), object(5)

memory usage: 86.2+ KB

Q2. Check for missing values in the dataset. Show the count of null values for each column.

Program screenshot:

```
#Q2
print("Missing Values in Each Column:\n")
print(df.isnull().sum())
```

```
*** Missing Values in Each Column:
```

```
Age          0
Sex          0
ChestPainType 0
RestingBP    0
Cholesterol  0
FastingBS    0
RestingECG   0
MaxHR        0
ExerciseAngina 0
Oldpeak      0
ST_Slope     0
HeartDisease 0
dtype: int64
```

Q3. Check for duplicate rows in the dataset. If any duplicates are found, remove them and print the new shape of the dataset.

Program screenshot:

```
#Q3
duplicates = df.duplicated().sum()

print("Number of Duplicate Rows:", duplicates)

if duplicates > 0:
    df = df.drop_duplicates()
    print("Duplicates Removed.")

print("New Shape:", df.shape)
```

```
*** Number of Duplicate Rows: 0
New Shape: (918, 12)
```

Q4. Identify unrealistic/invalid values:

- Count how many rows have Cholesterol = 0.
- Count how many rows have Resting BP 0.

Print both counts.

Program screenshot:

```
#Q4
chol_zero = (df['Cholesterol'] == 0).sum()
bp_zero = (df['RestingBP'] == 0).sum()

print("Rows with Cholesterol = 0 :", chol_zero)
print("Rows with RestingBP = 0 :", bp_zero)
```

```
... Rows with Cholesterol = 0 : 172
Rows with RestingBP = 0 : 1
```

Q5. Clean the invalid values:

- Replace Cholesterol = 0 with the mean cholesterol value (excluding zeros).
- Replace Resting BP 0 with the mean resting blood pressure value (excluding zeros).
- Round both columns to 2 decimal places.

Print the statistical summary (describe()) of these two columns before and after cleaning.

Program screenshot:

```
#Q5
print("Before Cleaning\n")
print(df[['Cholesterol', 'RestingBP']].describe())

chol_mean = df[df['Cholesterol'] != 0]['Cholesterol'].mean()
bp_mean = df[df['RestingBP'] != 0]['RestingBP'].mean()

df['Cholesterol'] = df['Cholesterol'].replace(0, chol_mean)
df['RestingBP'] = df['RestingBP'].replace(0, bp_mean)

df['Cholesterol'] = df['Cholesterol'].round(2)
df['RestingBP'] = df['RestingBP'].round(2)

print("\nAfter Cleaning\n")
print(df[['Cholesterol', 'RestingBP']].describe())
```

*** Before Cleaning

	Cholesterol	RestingBP
count	918.000000	918.000000
mean	244.636253	132.540893
std	53.318029	17.989932
min	85.000000	80.000000
25%	214.000000	120.000000
50%	244.640000	130.000000
75%	267.000000	140.000000
max	603.000000	200.000000

After Cleaning

	Cholesterol	RestingBP
count	918.000000	918.000000
mean	244.636253	132.540893
std	53.318029	17.989932
min	85.000000	80.000000
25%	214.000000	120.000000
50%	244.640000	130.000000
75%	267.000000	140.000000
max	603.000000	200.000000

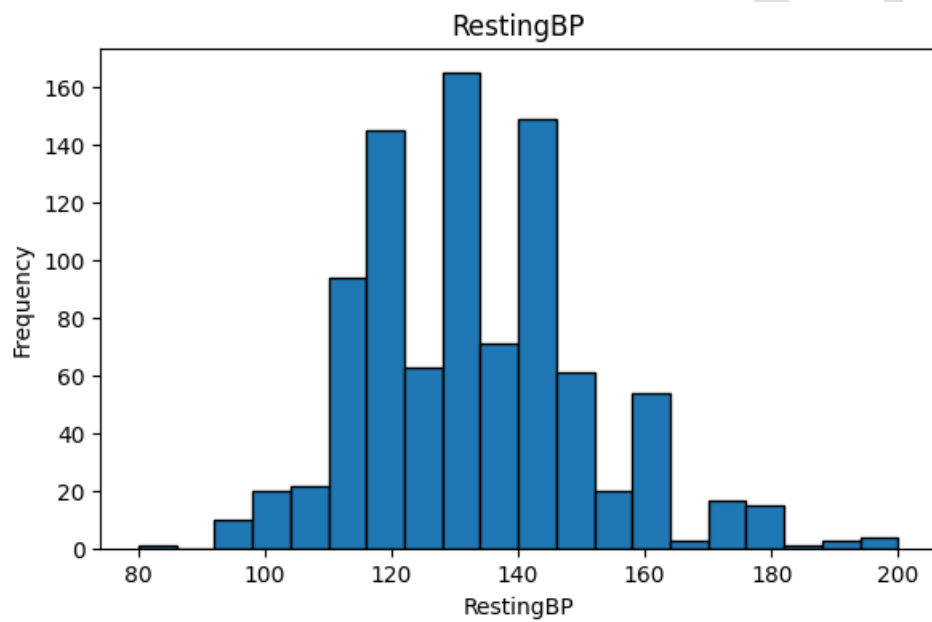
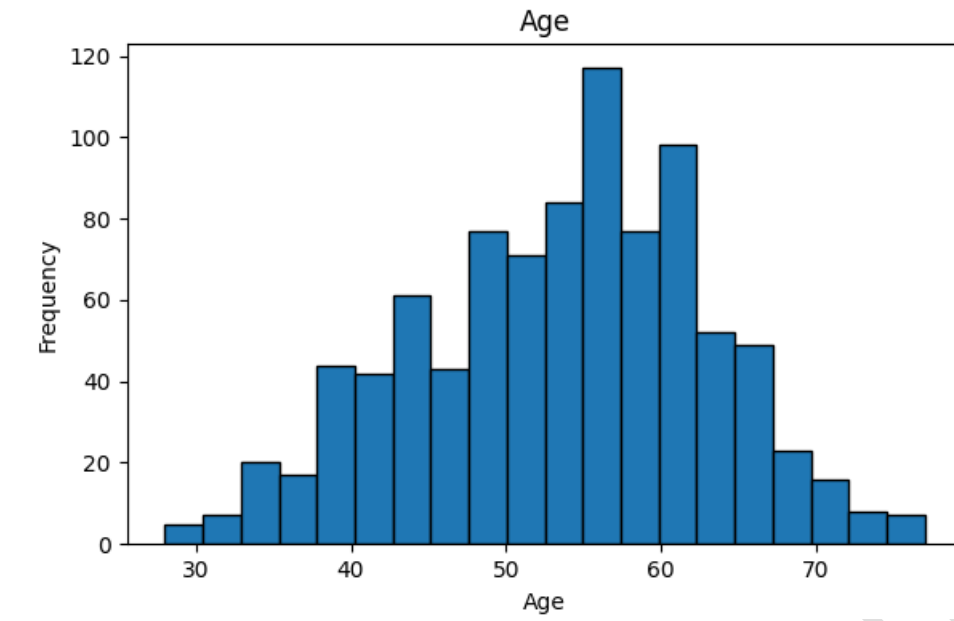
Q6. Create a function to plot histograms for the following numerical columns:

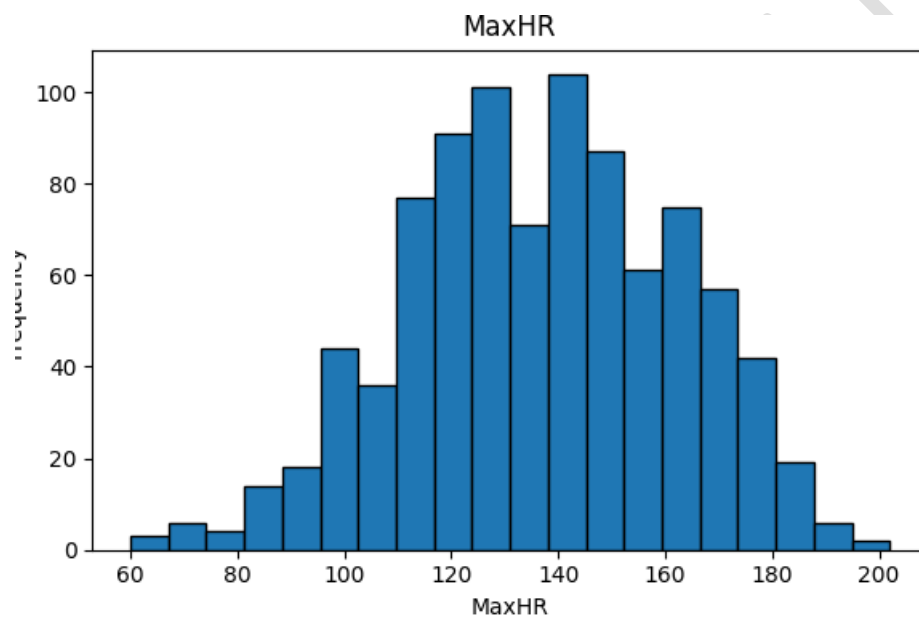
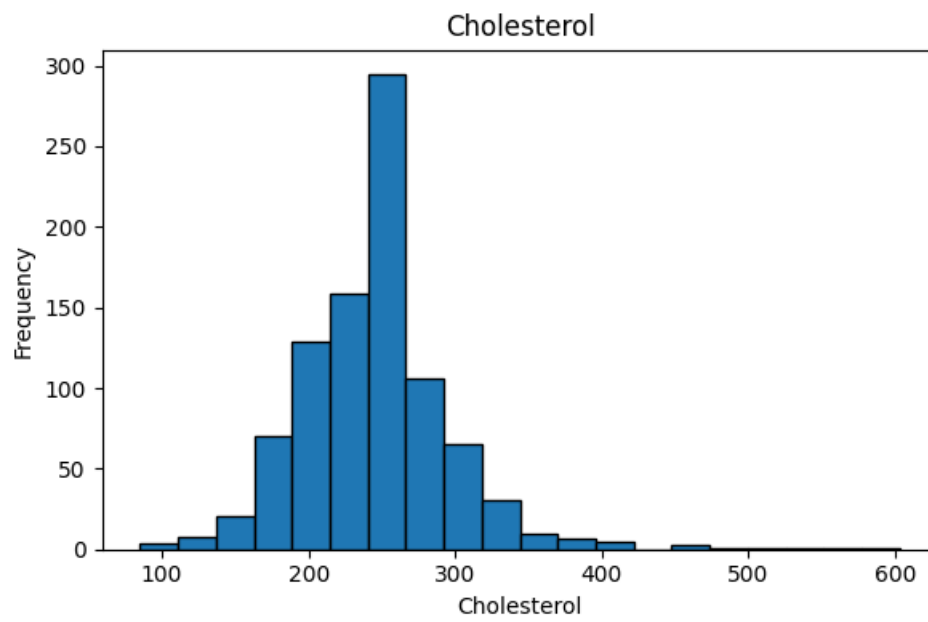
- Age
- Resting BP
- Cholesterol
- Max HR

Plot all four histograms in one figure using subplots 2 2 layout). Use this function to visualize the data after cleaning.

Program screenshot:

```
1  #Q6
   def plot_histograms(data):
       columns = ['Age', 'RestingBP', 'Cholesterol', 'MaxHR']
       plt.figure(figsize=(12,8))
       for i, col in enumerate(columns):
           plt.subplot(2,2,i+1)
           plt.hist(data[col], bins=20, edgecolor='black')
           plt.title(col)
           plt.xlabel(col)
           plt.ylabel("Frequency")
       plt.tight_layout()
       plt.show()
       plot_histograms(df)
```





Q7. Identify and print numerical columns and categorical columns separately.

Program screenshot:

```
#Q7
numerical_columns = df.select_dtypes(include=['int64', 'float64']).columns

categorical_columns = df.select_dtypes(include=['object']).columns

print("Numerical Columns:")
print(list(numerical_columns))

print("\nCategorical Columns:")
print(list(categorical_columns))

*** Numerical Columns:
['Age', 'RestingBP', 'Cholesterol', 'FastingBS', 'MaxHR', 'Oldpeak', 'HeartDisease']

Categorical Columns:
['Sex', 'ChestPainType', 'RestingECG', 'ExerciseAngina', 'ST_Slope']
```

Q8. Perform One-Hot Encoding on all categorical columns using `pd.get_dummies()`. Store the result in a new dataframe called `df_encoded`. Print the shape and the first 5 rows of the encoded dataframe.

Program screenshot:

```
#Q8
df_encoded = pd.get_dummies(df, columns=categorical_columns)

print("Shape of Encoded DataFrame:")
print(df_encoded.shape)

print("\nFirst 5 Rows:")
print(df_encoded.head())
```


Shape of Encoded DataFrame:
(918, 21)

First 5 Rows:

	Age	RestingBP	Cholesterol	FastingBS	MaxHR	Oldpeak	HeartDisease	\
0	40	140.0	289.0	0	172	0.0	0	
1	49	160.0	180.0	0	156	1.0	1	
2	37	130.0	283.0	0	98	0.0	0	
3	48	138.0	214.0	0	108	1.5	1	
4	54	150.0	195.0	0	122	0.0	0	

	Sex_F	Sex_M	ChestPainType_ASY	...	ChestPainType_NAP	ChestPainType_TA	\
0	False	True	False	...	False	False	
1	True	False	False	...	True	False	
2	False	True	False	...	False	False	
3	True	False	True	...	False	False	
4	False	True	False	...	True	False	

	RestingECG_LVH	RestingECG_Normal	RestingECG_ST	ExerciseAngina_N	\
0	False	True	False	True	
1	False	True	False	True	
2	False	False	True	True	
3	False	True	False	False	
4	False	True	False	True	

	ExerciseAngina_Y	ST_Slope_Down	ST_Slope_Flat	ST_Slope_Up
0	False	False	False	True
1	False	False	True	False
2	False	False	False	True
3	True	False	True	False
4	False	False	False	True

[5 rows x 21 columns]

Q9. After cleaning and encoding:

- Print the final shape of df_encoded.
- Show the list of all column names in the final dataframe

Program screenshot:

```
#Q9
print("Final Shape:")
print(df_encoded.shape)

print("\nColumn Names:")
print(df_encoded.columns.tolist())

... Final Shape:
(918, 21)

Column Names:
['Age', 'RestingBP', 'Cholesterol', 'FastingBS', 'MaxHR', 'Oldpeak', 'HeartDisease', 'Sex_F', 'Sex_M', 'ChestPainType_ASY', 'ChestPainType_ATA',
```

```
#Q9
print("Final Shape:")
print(df_encoded.shape)

print("\nColumn Names:")
print(df_encoded.columns.tolist())
```

```
, 'RestingECG_LVH', 'RestingECG_Normal', 'RestingECG_ST', 'ExerciseAngina_N', 'ExerciseAngina_Y', 'ST_Slope_Down', 'ST_Slope_Flat', 'ST_Slope_Up']
```

Q10. (Summary)

1. Write a short summary (in comments or markdown) covering:
2. What invalid values did you find and how did you fix them?
3. Why is it important to handle invalid values like Cholesterol = 0?
4. What is the purpose of One-Hot Encoding? Any other observations from the cleaning process

```
#Q10
"""Summary
```

1. Invalid values found:

- Cholesterol = 0
- RestingBP = 0

2. Cleaning Method:

- Replaced Cholesterol = 0 with the mean cholesterol value (excluding zeros).
- Replaced RestingBP = 0 with the mean resting blood pressure (excluding zeros).
- Rounded both columns to two decimal places.

3. Importance of Handling Invalid Values:

- Invalid values can reduce model accuracy.
- They can distort statistical analysis and visualizations.
- Cleaning improves data quality and machine learning performance.

4. Purpose of One-Hot Encoding:

- Converts categorical variables into numerical format.
- Makes data suitable for machine learning algorithms.
- Prevents models from treating categories as ordinal values.

5. Other Observations:

- Duplicate rows were removed if present.
- Missing values were checked.
- Histograms were plotted after cleaning.
- Final dataset was encoded and ready for machine learning.